# CSI 2110 Data Structures and Algorithms (3 units)
# Spring/Summer 2026

**Professor:** Mohamed Ali Ibrahim, Ph.D., P. Eng.
**Office:** STE 468
**Email:** mibrahi3@uottawa.ca
**Office Hours:** By appointment

**Teaching assistants**: Zhuoran Xu, zxu045@uottawa.ca
Mithila Sivakumar, msiva025@uottawa.ca

## CSI 2110 Data Structures and Algorithms

The concept of abstract data types. Simple methods of complexity analysis. Trees. The search problem: balanced trees, binary-trees, hashing. Sorting. Graphs and simple graph algorithms: traversal, minimum spanning tree. Strings and pattern matching.
**Course Component:** Laboratory, Lecture, Tutorial
Prerequisites: ITI 1121, MAT 1348.

## Lectures/Labs/Tutorials

| Section | Activity | Capacity | Schedule | Room | Dates |
|---|---|---|---|---|---|
| CSI 2110-Z00 | **Lecture**: Data Structures and Algorithms | 48 | Tuesday 10:00 AM – 11:20 AM | 65 University — MRT 212 | May 4, 2026 – July 24, 2026 |
| CSI 2110-Z00 | **Lecture**: Data Structures and Algorithms | 48 | Thursday 8:30 AM – 9:50 AM | 65 University — MRT 212 | May 4, 2026 – July 24, 2026 |
| CSI 2110-Z01 | Laboratory: Data Structures and Algorithms | 23 | Friday 1:00 PM – 2:20 PM | 800 King Edward — STE 2060 | May 4, 2026 – July 24, 2026 |
| CSI 2110-Z02 | Laboratory: Data Structures and Algorithms | 25 | Monday 10:00 AM – 11:20 AM | 800 King Edward — STE 2060 | May 4, 2026 – July 24, 2026 |
| CSI 2110-Z03 | Tutorial: Data Structures and Algorithms | 48 | Tuesday 11:30 AM – 12:50 PM | 65 University — MRT 205 | May 4, 2026 – July 24, 2026 |

**Course outline** (the dates proposed here are indicative only and subject to change)

| Lecture | Date | Topic | Textbook chapters |
|---|---|---|---|
| 1 | May 5 | Course introduction; ADTs; review of arrays and linked structures | Ch. 3 |
| 2 | May 7 | Algorithm analysis; growth rates; Big-O notation | Ch. 4 |
| 3 | May 12 | Complexity examples; simple justification techniques | Ch. 4 |
| 4 | May 14 | Recursion review for data structures and algorithms | Ch. 5 |
| 5 | May 19 | Stacks and stack applications | Ch. 6 |
| 6 | May 21 | Queues, circular queues, and deques | Ch. 6 |
| 7 | May 26 | List ADT, array lists, dynamic arrays | Ch. 7 |
| 8 | May 28 | Positional lists, iterators, Java Collections Framework | Ch. 7 |
| 9 | June 2 | Trees: terminology, Tree ADT, general trees | Ch. 8 |
| 10 | June 4 | Binary trees and tree traversals | Ch. 8 |
| 11 | June 9 | Review for midterm; problem-solving session | Ch. 3–8 |
| 12 | June 11 | **Midterm Exam** | Ch. 3–8 |
| 13 | June 16 | Priority queues and heaps | Ch. 9 |
| 14 | June 18 | Maps and hash tables | Ch. 10 |
| 15 | June 23 | Hash functions, collision handling, load factors | Ch. 10 |
| 16 | June 25 | Binary search trees | Ch. 11 |
| 17 | June 30 | Balanced search trees: AVL trees and red-black tree concepts | Ch. 11 |
| 18 | July 2 | Sorting I: merge-sort and divide-and-conquer | Ch. 12 |
| 19 | July 7 | Sorting II: quick-sort, heap-sort, and sorting comparison | Ch. 12 |
| 20 | July 9 | Strings and pattern matching | Ch. 13 |
| 21 | July 14 | Graph ADT and graph representations | Ch. 14 |
| 22 | July 16 | Graph traversals: DFS and BFS | Ch. 14 |
| 23 | July 21 | Minimum spanning trees: Prim-Jarník and Kruskal | Ch. 14 |
| 24 | July 23 | Final review and integration of course concepts | Ch. 3–14 |

**Learning Outcomes**

By the end of this course, students will be able to:

1. Explain and use fundamental data structures, including arrays, linked structures, stacks, queues, deques, lists, trees, priority queues, maps, hash tables, search trees, and graphs.
2. Apply abstract data types (ADTs) to model and solve computational problems using appropriate interfaces, operations, and implementation strategies.
3. Analyze the efficiency of algorithms using growth rates, asymptotic notation, and Big-O analysis.
4. Compare algorithms and data structures based on time complexity, space complexity, implementation cost, and suitability for a given problem.
5. Design and implement recursive algorithms and explain how recursion can be used in data structure operations and algorithmic problem solving.
6. Implement and use linear data structures such as stacks, queues, deques, array lists, positional lists, and iterators in Java.
7. Implement and traverse tree-based data structures, including general trees, binary trees, heaps, binary search trees, AVL trees, and red-black tree concepts.
8. Use hashing techniques to implement maps and dictionaries, including hash functions, collision handling, and load factor management.
9. Implement and compare sorting algorithms, including merge-sort, quick-sort, and heap-sort, and explain their performance characteristics.
10. Apply string-processing techniques, including basic pattern matching algorithms.
11. Represent and process graphs using appropriate graph ADTs, adjacency structures, and traversal algorithms such as depth-first search and breadth-first search.
12. Apply graph algorithms to solve problems involving connectivity, traversal, and minimum spanning trees, including Prim-Jarník's and Kruskal's algorithms.
13. Select appropriate data structures and algorithms for practical software problems and justify the choice using correctness, efficiency, and maintainability.
14. Develop Java programs that implement core data structures and algorithms while following good programming practices, modular design, and clear documentation.

**Textbook:**

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. *Data Structures and Algorithms in Java*, 6th Edition. Wiley.
Edition Paperback – Jan. 28, 2014
Cost: $212.40.

**Reference books:**

1. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. Useful as a strong reference for algorithm design, analysis, sorting, searching, graphs, and strings.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
   A rigorous reference for algorithm analysis, asymptotic notation, recurrences, dynamic programming, greedy algorithms, graphs, and advanced topics.
3. Weiss, M. A. (2012). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson. A useful Java-based reference for lists, stacks, queues, trees, hashing, priority queues, sorting, and graph algorithms.
4. Shaffer, C. A. (2013). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Dover Publications.

A complementary reference with explanations of algorithm efficiency, data structures, and implementation issues in Java.

5. Lafore, R. (2002). *Data Structures and Algorithms in Java* (2nd ed.). Sams Publishing. A more accessible reference with many practical Java examples, useful for students who need additional explanation.

**Grading Scheme:**

| | Grade |
|---|---|
| Participation quizzes | 10 % |
| Laboratories (5) | 20 % |
| Mid-term Exam | 25 % |
| Final Exam | 45 % |

**Participation quizzes**:
A participation quiz will be administered during each class session through **Brightspace**. The grade for each quiz will be based on two components: **75% for participation** and **25% for the accuracy of the response**.

These quizzes are not formal assessments; rather, they are participation-based activities designed to encourage active engagement, regular attendance, and active learning during class.

**Laboratories**:
I firmly support a learning-by-doing pedagogical approach. In alignment with this methodology, students will complete one laboratory assignment every two weeks, resulting in a total of five laboratory sessions over the course of the semester. Students will work in teams of two to three

A formal laboratory report is required from each group following every laboratory session. Reports must be submitted electronically via Brightspace at https://uottawa.brightspace.com/.
Late submissions will not be accepted under any circumstances.
**Midterm exam**: Closed-book exam on the material studied in the first part of the course, on Thursday June 11.

**Final Exam**: Closed-book exam covering all the material studied in the course.

**Note:** Participation quizzes, laboratories, and tutorials will begin in the **second week of the term**.
.

**General Policies and Procedures**

- Plagiarism and Academic Integrity
  - Zero percent for work in which cheating is detected.
  - Other penalties (including course failure and expulsion) are also possible.
  - See https://www.uottawa.ca/about/sites/www.uottawa.ca.about/files/plagiarism.pdf
  - or https://www.uottawa.ca/about/sites/www.uottawa.ca.about/files/plagiat.pdf
- Academic Fraud
  - See the following link for important information:
    http://www.uottawa.ca/academic/info/regist/crs/0305/home_5_ENG.htm
    http://www.uottawa.ca/academic/info/regist/crs/0305/home_5_FR.htm
- Course Policies

- Zero percent for late work.
- In a 4th-year course, it is unreasonable to expect that all testable material will be covered in class (real life is not like this!). Make use of the suggested textbooks and any other relevant material.
- Electronic devices are not to be used in class without explicit permission from the instructor.

- University of Ottawa Academic Regulations
  https://www.uottawa.ca/administration-and-governance/policies-and-regulations
  https://www.uottawa.ca/administration-et-gouvernance/politiques-et-reglements
- Specific University of Ottawa Academic Regulations
  - Class attendance is mandatory. As per academic regulations, students who do not attend at least 80% of the classes may not be allowed to write the final examination.
  - Students must be registered in order to take the class. Students who are not officially registered cannot receive a mark.
  - All components of the course (i.e., assignments, exams) must be fulfilled otherwise students may receive an INC as a final mark (equivalent to an F). This also holds for students taking a course for the second time.